

Course Overview

Course Overview

Hi, everyone. My name is Steve Smith, a.k.a. Ardalis, and welcome to my course, Refactoring to SOLID C# Code. I've been working with and teaching students how to be productive with .NET since the first versions of C# and .NET shipped. I published my first Pluralsight course in 2010, also on SOLID principles. And my company, NimblePros, helps teams write better code faster. And that's really my motivation for sharing tips and techniques with you here on Pluralsight too. Believe me, we have seen some things, but we usually can help improve even the worst cases of legacy code. This course will teach you how to apply SOLID principles as you assess and refactor some ugly legacy code. The sample for this course was based on real-world code that I see from clients as I help them improve their systems. In this course, you'll learn how to do exactly that yourself. After a few passes, we're also going to assess the code using proven metrics to see if we've made a measurable improvement. Some of the major topics that we will cover include analyzing and assessing legacy systems, applying SOLID principles and refactoring to improve design, understanding unit testability and how to test legacy code with both unit and integration tests, and how to measure code quality improvements using static analysis tools. By the end of this course, you'll know how to improve legacy systems so they can be upgraded, supported, and extended for years to come. Before beginning this course, you should be familiar with .NET, C#, and perhaps ASP.NET. From here, you should feel comfortable diving into additional courses on domain-driven design, N-tier architecture or design patterns. I hope you'll join me on this journey to learn how to improve legacy systems with the Refactoring to SOLID C# Code course, here at Pluralsight.

Assessing Legacy Code

Introduction

Welcome, everyone, to this course on Refactoring to SOLID C# Code. I'm Steve Smith, a.k.a. Ardalis, and I'll be your guide. In this course, you're going to encounter legacy code with certain characteristics that make it more difficult than it should be to maintain, extend, and test. I'm going to show you how to tame large legacy codebases by analyzing and then gradually refactoring them into something better. In this first module, Assessing Legacy Code, we're going to examine some fairly complex code that's inspired by real-world software. We'll look it over together, gather some data about the code, and then review the results. Our goal is to have an understanding of what the code is doing, how it's organized, and where the seams are that we can start to use to improve it. Let's get started. Let's do a quick version check for this module. This course was created using the displayed versions of C#, .NET, and Visual Studio. This slide shows the versions of C# and .NET for which the information in this course applies. In this module, we are going to introduce a legacy code example inspired by real-world software. It will consist primarily of a complex pricing service, which plays several roles at the heart of a business. We'll review the code together. I'll act as part tour guide, part archaeologist as we examine the legacy code and try to determine what it does and why it's built the way it is. We'll leverage principles like SOLID, as we look for ways in which the code could be improved, and we'll identify some code smells for future attention. Then, we'll take what we've learned and develop a plan. When you're refactoring some code you just wrote 5 minutes ago, you don't really need much planning. But when you're refactoring complex legacy code that's mission-critical to the business like a pricing calculator, planning is an important part of the process. As part of the plan, we'll try to identify low-hanging fruit, things that we can do with minimal risk and/or effort that will yield the biggest gains in code quality and maintainability.

Introducing the Sample App

To begin, we're going to need some example code. I've chosen to use a domain I expect many of us are familiar with, home remodeling. Specifically, since my kitchen is currently mid remodel, I figured we would look at some code that a kitchen remodeling company might use to help price new kitchens. The main function we're going to be looking at is called `CalculateTotalPrice`. When determining the price for a given part of a kitchen remodel, the software breaks down the project into individual walls. Within a given wall, there will be one or more cabinets. We'll also have to deal with things on the wall where there are no cabinets. Then, within a cabinet, there may be some number of features. Features

is just a generic term for things like doors, shelves, handles, knobs, etc. As you can probably guess, there are some exceptions to this taxonomy. What about islands and peninsulas? What wall would these be on? Well, some walls are really just virtual containers for cabinets and appliances, not literal walls, but that's how the original function was written and how it has evolved today. As we'll see, there are checks to see if something is, for example, an island. And in such cases, there may be some custom logic. The `CalculateTotalPrice` method can also be run in one of three different modes based on a parameter that is passed into it. It can show a real-time quote on the kitchen builder web application. This is the default behavior and is used to reprice a kitchen remodel every time changes are made. It can also be used to place an order for the current configuration. Once the customer has agreed to move forward, this is the functionality that creates the order. Finally, it can generate a text report, summarizing the details of the configuration and its components. This may include specific information about customization in addition to serving as a parts list for the contractor. In addition to the parts and quantities, it also contains information about pricing and measurements.

Demo: A Tour of the Sample App

Okay, let's dive in. Now, before we start, I want to mention that this is some pretty complicated code. I'm not expecting you to fully understand it without first downloading the sample code and going through it yourself, and I don't really expect you to even do that. But I do want you to be able to follow along with how I go about analyzing the code so that you know what to look for yourself. So, what should we be looking for? We want to get a sense of the structure of the code. Where does it live? What are its inputs? What calls it? What does it return? We want to figure out how the code fits into the larger application, what other code calls it, what parts of the code does it call and especially what are its tight coupling points? For those, we'll be especially wary of static code and instances of the new keyword. Of course, we'll want to look for any code smells that jump out at us. I'm not going to give you a list of code smells here. I discuss them in some of my other refactoring courses, but I'll point out some we see along the way. There are sure to be some violations of SOLID and related principles, which we will point out. And any issues we find with the code organization, complexity, maintainability or testability will all be worth noting as well. That's a lot to consider, but let's go ahead and dive into the code now. Okay, here we have the application loaded up in Visual Studio. I'm going to spend most of the time looking at the code, but I want to initially just orient you to the solution. This

is a simple ASP.NET Core web application. It's using Razor Pages just because it's a sample, but imagine in a real application there would be some kind of a Blazor or complex JavaScript front end that would help you with producing, and visualizing, and choosing all the options for a kitchen remodel. In this case, I'm specifying all of the data inside of the SeedData.cs file you see here at the bottom. Then, inside of the index.cshtml, we're going to make a call to our pricing service, and that's going to calculate the price for the chosen options that I have for my kitchen. Then, if we want to use the generate file report option or the place order option, those will be called from the GenerateFile.cshtml or PlaceOrder.cshtml files. In fact, let's look at those first. To generate a file, we are just going to call PricingService.CalculatePrice, pass in a few values and then specify that it's a PriceReport. Everything else that you see here is just setting up some stuff in the session, and that's done for each one of these different pages because we're using session values inside the report. Similarly, for placing an order, it looks pretty much the same. The only difference is that last parameter is set to Order. I'm going to give you a tour of the PricingService class. I want you to think about me as if I'm a developer showing you this code that maybe I've worked on. And since you're not here to do it, I'm going to also act as you analyzing that code and noting things that you might discover or think about. Maybe I'll mention some things that you wouldn't have found or that you wouldn't have cared about, and maybe I'll miss some things that you would've wanted to change. As we look at the code, I want you to think about and maybe even write down all the things that you would have an issue with and that you would think would be worth considering changing or refactoring in this code. All right, so without further ado, here is the pricing service code. This is CalculatePrice. It is a static method. It takes in four different parameters, the kitchenId, the wallOrderNum, which is the numeric sequence of the wall within the kitchen, the userName, and a refType. We already saw what that last parameter is used for, which is helpful because if you look at it right here, refType doesn't really tell you a whole lot about what that's actually used for. Let's just look at the next line. Very first off thing we're going to do is some sort of a guard clause that's going to say if pricing is off, if that is set to a single character string with a capital Y, then we're going to return 0|0|0. Notice, CalculatePrice is returning a string. I'm not sure I love that. And it also seems to be a string that's composed of multiple values. I don't really know what those are at first glance here, but what we're going to learn. And if I were the developer showing you this code, I would tell you that that is the base price, the price with some markup for the company, and potentially the price with some additional markup for the salesperson that is actually

demoing this to a customer. Right now, we're going to new up a kitchen, new up an order and set a whole lot of floats to different values. And we're just going to keep on scrolling here. We have a lot of local values that we're going to set here. And some of these, my IDE is telling me that this value is assigned, but its value is actually never used. So, these are some things that I would definitely add to my list of items that probably could get refactored away. The other thing that jumps out at me here is a couple of things. One, these are floats, but a lot of them appear to be used for monetary values, subtotal and grandtotal. I have to believe those are going to be price-related, and, in fact, they are. It's much better to use a decimal for that than a float. You'll avoid rounding errors among other things, so I would want to change that type if I can. And then the other thing is I really don't like declaring all my variables at the top of the method, especially for a method as long as this one. And we haven't even looked at how big it is yet, but you can see from the scrollbar on the right that it's fairly long. These values, a lot of them are unused because they're all declared at the top, not where they're going to be used. And so it makes it so there's a big disconnect in vertical distance in your code between when they're declared and when they're used, making it more difficult for you to figure out what the heck they're for. Better still is to use shorter methods and to make sure that you're using the variable near where you're assigning it and declaring it. Ideally, you should declare it and assign it in the same place. At least it's doing that. It looks like we're also going to set a few data tables here at the end. So, there's three different data tables. They have great names, that's sarcasm, dt, dt2, dt3, and a StreamWriter, sr. Three different data tables all called just dt and then some number is not going to be very helpful. So we'd want to give those better names or, like I said, just declare them where we need them and give them the appropriate name for the data they're going to hold. Now it looks like we're going to assign some value to Session. Not sure I like that. Static access that has side effects in this method, it may make it more difficult to change this class relative to other classes that might depend on that value. All right, so the side effects of that session access, especially since we're writing, not just reading, could have far-reaching effects within this application. We have to be super careful about that kind of code when we're refactoring. We're going to start a try catch block. And in here, we're going to do a few more guard clause checks. We're going to see if the wallOrderNum is equal to 0, then we imagine that the session has expired and we tell them to log in again. If the kitchenId is less than or equal to 0, we'll just tell them it's invalid. Then we're going to tell the kitchen to load or GetCustomerKitchen, passing in an Id and a userName. Let's see, what does that do? All right, so

right in here now, this is inside the Kitchen.cs class, we're going to assign the kitchenId to the property on this class right here. And then we're going to use a SQLite connection to create a command with a hard-coded SQL string right here, selecting the various columns from kitchen. Make sure the UserId is the right UserId and do a DataReader and then assign some more properties. All right, so I don't necessarily love this pattern. It's a little bit like active record, but not really because we're not newing up the kitchen. We newed up the kitchen higher up, and then here we're just loading it with data. So, until this point, we had an instance of a kitchen that wasn't fully populated, but we could have started using it in its invalid state. Now presumably, after we have called this line, it's in a state where we can actually use it. Then we're going to set these couple of variables here, the bbHeight and bbDepth. I don't really know what those are, so we'll see if they end up being used for anything. And then we're going to create another SQL connection, and this time we're going to make a query to get some walls from the kitchen, and those are just going to go into a data table. So we're going to get a DataReader, and we're going to load it into a data table. And we're going to say if we didn't get any rows, then it was invalid. Otherwise, we're going to say if we're doing a PriceReport, that's interesting, now we're going to apparently start creating a file and set the baseDirectory, create a file called Orders.csv, create that StreamWriter for sr. Sr should be a StreamReader, I guess. Now that I'm thinking it's a StreamWriter, it probably should be sw, shouldn't it? And then we're going to write out what looks like the headers of this report. Otherwise, if it's an order, let's scroll down here, if it's an order, we're going to create a new order, this helpful comment tells us. And again, we're going to just open up a connection and do an INSERT now into the ORDERS table with the KitchenId, OrderDate, OrderStatus, OrderType. And then we'll grab the rowid so we can set the OrderId. Now after all those if statements, now we're going to set some values here on this section right here. These are all related to that dt. And recall that that dt was the walls. And so we're getting the first row of that data table full of walls, and we're going to grab the color, the wallId, whether or not it's an island, and its height. All right, then we're going to go and do another SQL statement. We're going to select all the cabinets for this wallId and load that into a data table. Now, we're already using dt, so naturally we're going to use dt2 for this. And so then we're going to loop through each row in dt2, and we're going to grab a bunch of data from there to see what are all the things that have to do with that cabinet. So each one of these rows is actually a cabinet, and each one of these variables here, we have a locally declared cabinetId, but then we have these other ones that were declared up at the top, the PartWidth, and Depth, and

Height, and Color, and Sku, and Cost. All those things are variables that we declared way up at the top of this method. And we're going to assign all those to cabinet values. So, when we say thisPart, I guess we mean this cabinet part, at least in this case. Then, as long as we have looks like some kind of a SKU, a SKU is a stock keeping unit, we're going to go and grab from all the pricing SKUs it looks like in the database, we're going to see if there's a row for this SKU. And if there is, if we get a row and we can read, then we're going to assign the cost to be the WholesalePrice. Normally, I'd like for these two things to match in their naming, but I can see what this is doing. All right. If not, then thisPartCost is going to have some other value, which we set to 0 here, so we at least initialized it there. That's good. Then we're going to go get PricingColors with the PricingColorId, which is going to be thisPartColor. So apparently, thisPartColor isn't actually a color. It's actually just an integer value. It's just an ID. Don't love that, that we name it Color here and not Id, but that's fine. Here, it's an Id all of a sudden. And then if we do find one of those, we're going to set the ColorName, Markup, and ColorSquareFoot. I guess it's ColorPerSquareFoot, so I guess that's a price of some sort. And then we're going to set some of these subtotal things. So, here we're going to get the TotalPartCost is equal to the PartCost times the ColorMarkup percentage plus 1 divided by 100. So, if ColorMarkup is 5, then this would make it be a 5% increase. We're going to increment the subtotal by the TotalPartCost. We're going to implement subtotalFlat by just the PartCost, so without this markup. All right, so the flat cost is without the markup. Let's keep going here. So now we're going to go grab the UserMarkups from a SQL table, and we'll use another DataReader here, and we'll just grab thisUserMarkup to be the MarkupPercent. And then subtotalPlus, which is a third subtotal variable that we're tracking, is going to be the TotalPartCost times thisUserMarkup divided by 100 + 1. So again, if the user markup is 5, then this will be an additional 5% on top of whatever markup there might already have been up here. All right, and that's going to be in subtotalPlus. Now if we are creating an order, then this is going to be where we add this item to the order. So this cabinet that we're on, we have to buy that cabinet. So we're going to insert an ORDERITEM with this SKU for this cabinet and specify the price and all the other things here, and then do that INSERT statement. But if this is a PriceReport, then we're going to write out a new line and specify all these different variables to say this is the thing that we're trying to get. Otherwise, we just get the cost. I don't know why we have this else statement here. That could probably go. Then we're onto features. Remember, I mentioned that we have a kitchen that has walls, which have cabinets, which have features. We have just now gotten to where the features on

this cabinet exist, if any. So we're going to select all those features. We're going to load them into a data table. Now we've already used data table 2 and dt, so, of course, this will be dt3. And in here now, we're going to set a bunch of local variables. Notice we didn't have to set these at the top. Somebody came along later I guess and just decided that things that are scoped only within this for each loop made sense for feature. So we have a featureId, colorId, SKU, etc. We're going to assign all those things here. And then if we have a colorId, we're going to go back out to PricingColors. Pass it that colorId, and get a whole bunch of data here. Looks like we're grabbing the name, the markup, the square foot cost, the wholesale price. and there's a couple of different ways we're doing pricing here. I see pricing per square foot, some kind of markup, some kind of wholesale price. So the area in square foot is the height times the width divided by 144 because those must be in inches. So, a square foot is 12 x 12 is 144. So that makes sense, but I don't like the magic number. FeatureCost is then just going to be the area times the square foot. Color cost, all right, I'd probably rename that, but that's fine. If the featureCost is 0, that means we must not have any area or we didn't specify a color square foot cost. Then we're going to use the wholesalePrice times the quantity. Okay, so we either have an area base price or an item base price, I guess is what we're taking away from this. And then we're going to do the same kind of math here to figure out the featureCost, the subtotal, subtotalFlat, subtotalPlus. All right. If it's an order, then we're going to add it to the order. If it's a report, then we're going to add a line to the report. If it's not an island, we're going to price the wall color backing around the cabinets. So, we're going to go grab the pricing colors for the default color, which we set way up at the top when we loaded the wall. And then in here, we're going to find some paint with some markup for some square footage. and we're going to figure out how much it costs to paint the wall, I guess. And then we'll insert that into the ORDERITEM if it's an order, otherwise we will add it to our price list. And then when we're done, we're going to do a String.Format to return our data with the subtotal, the subtotalFlat, the subtotalPlus. If anything goes wrong, we're going to just send an email and throw. I like that we're at least not throwing a new exception, but we're throwing the one that we have. And then we're going to clean up a little bit and clean up that sr and make sure it gets disposed. That's a lot, but that's what we're looking at.

What Did We Find?

Let's think about the problems we discovered and how we can go about starting to fix them. The first step is to consider the broad categories of problems we found. Then, we can make a list of the specific problems and start to prioritize addressing them. One big thing we found was repetition. This code does not follow the Don't Repeat Yourself, or DRY, principle very well. A lot of the repetition involved how data access was performed, as well as certain conditional statements. Speaking of data access, this code had many different abstraction levels. There's high-level business code for calculating prices and markups, and it's sitting side by side with low-level code for talking to a particular database or writing to a specific output file and format. And there is no abstraction, so there's no way to follow the open-closed principle or to invert dependencies and use the dependency inversion principle. There's very little use of object-oriented code or even data types. Everything is done with primitive local variables with the exception of some data table and DataReader usage. Even classes like Kitchen are just anemic bags of properties. There is a lot of complexity. In some places, we were indented in at least 10 times. I set my editor to only use two spaces per indent just so I could show you the code effectively, and it was still indented pretty far. In fact, if we use Visual Studio to calculate code metrics for the project and drill into the pricing service, we can see that it has a cyclomatic complexity score of 40. That's very high for a single method. It should be rare to have methods with more than a complexity level of 10. Of course, that high complexity is at least partly because this class or really this method has way too many responsibilities. It is definitely not following the single-responsibility principle. And finally, as a result of the heavy use of statics and the lack of abstractions, this code is very tightly coupled to its infrastructure, as well as how it's performing all of its business logic.

Demo: Recording Problems with the Sample App

In this demo, we will look at the code a second time, but this time with an eye toward identifying problems, not just reviewing how the code works. I'm going to move quickly this time so we can wrap up this module with how to make a plan to address the issues we find. All right, so we're back inside Visual Studio. You can see the cyclomatic complexity numbers here that you get from the code metrics. You can also use other tools to get that same value. That's just for this one method. Now, let's go through quickly and see some of the problems that we might want to refactor. First, this method is static. That makes it much more difficult for us to work with. I would much rather have that be an instance method on an instance class so that we can do dependency injection and a bunch of other

things. Next, it returns a string value even though it's calculating a price. The real problem here is it's calculating three different prices. So, we really need this to return some type of an object that is a collection of three different values. I don't like the fact that we're using the string refType for this method, so we should have different methods to do different things probably. And we can either inject in the different behaviors into that one method, or we can have the calculation logic get injected into different methods for calculating the price, producing an order or doing a report. I don't like that we're using Session this way to track whether or not we want to actually get pricing data. I'm sure there's a better solution to this, so I would try and refactor away the Session logic. I already talked about how most of these things here should just go away. All the ones that are not used should just be deleted with prejudice. All the ones that are used should be refactored to use decimal, and they should also be declared near where they're being used. These data tables obviously should be renamed or removed entirely. The logic for populating of different data types should be in separate methods, and they should just return a strongly typed class or struct that has the information that we need. I already called out how Session access is very dangerous. In this case, we're not really using this for a whole lot, but we don't know what else might be. So it's dangerous for us to try and remove it lightly. We want to do some testing to make sure that this wasn't going to break anything and then get rid of it or find another way to do that coordination. The try catch logic is pretty straightforward, but it is a separate responsibility. I would consider moving that to a wrapper method around this code possibly using a decorator or something similar. These guard clauses could possibly be a little bit simpler, or we could just use some type of fluent validation. Also, since we're going to change the return type, it might make sense to use a result abstraction here so that in the event that it isn't a successful return type, we could instead just investigate the result and see if it had any validation errors or other problems and then pull that data out of there. I'm not really a fan of the kitchen loading itself like this. I would much rather use a separate method rather than putting that responsibility on kitchen. It would be nice if kitchen had some responsibilities, but I don't think getting itself from persistence and being tightly coupled to SQLite, in this case, within that entity is the way to go. So I would pull that out into another method. Probably, ultimately, I would use a repository abstraction. But if we were using Entity Framework, I would be fine to just use DbContext to pull that out. All these calls here where we have a using statement, grabbing a connection and talking to SQLite to issue some type of a query, I would replace all of these with some type of helper methods. This could also be further improved by using

something like Dapper, which would reduce the amount of code required for this significantly. Also, instead of using standard data tables, this is code that was probably written 15 or 20 years ago. This type of code today, we would probably use just Dapper into a strongly typed result type. Or if we need to get multiple rows of data, maybe we would use Dapper coming into an IEnumerable or something like that. But Dapper or Entity Framework, either one of those is going to be a better choice than just doing this low-level code here. And we definitely don't want this low-level code to live in this method. Even if we're going to still use low-level ADO.NET, that code should live somewhere else. In here, we're doing some more checks. This, again, would probably be a result type. Here, we're doing some repeated logic on if types. I would probably start by trying to refactor that out and replace conditional with polymorphism. It's one of my favorite refactorings. What you would do is have a polymorphic way to perform this functionality. What that means is you'd have different types that each have a CalculatePrice method, and each one of those types would be implemented as a PriceReport class or an Order generator class or just PriceCalculator class. And so when you call CalculatePrice on the appropriate subtype, it'll perform this logic or it'll perform this logic based on which instance of that type you're calling. So basically, you can leverage inheritance. Alternately and even more flexibly, you can use the strategy design pattern and dependency injection to inject in a strategy for doing this logic inside of the outer shell that's doing the CalculatePrice. Here again, we have a lot of small, primitive values that are being used. These could all be added to a single type. If we do collapse this down to a much more compact class, it may make sense that these would be fields or properties on that class instead of having to be local variables inside the method. Again, another connection, we do the same thing, another loop over a data table. These would be strongly typed entities that we would be working with instead of just this collection of local variables inside of this method. Here, we're going to do another check that's going to have not one, but two different database queries coming with it. And so for this, we're trying to get all the data that we need in order to calculate the price. And then we're finally going to calculate the price here. Again, if we pull out those SQL calls, it should make this method much simpler. This is really the heart of this method. If we're going to say this method has a responsibility, it's this. It's calculating these price values and then returning them. So all the other logic of just trying to get it the data it needs are things that could be passed in or things where you could pass in a way to get that data either as a func or a service like a repository or some other data access service that could get the data that it needs. Here's yet another database call and a little bit more of

the business logic. When we're done, I would say that this price calculation logic here should be the only thing that's really left inside of the CalculatePrice method. Here, we have another example of checking the refType to see what type of thing this is. And then either inserting in an ORDERITEM or adding a line to the report. Again, this could be done with polymorphism. Obviously, this could just be deleted. And then when it comes time to doing the feature cost, it's more of the same. Take the SQL query, get rid of it, take the local variables that we're loading into from the rows on the data table. We don't really need to load this into a data table and then take each row and assign it to something. That could've just been a DataReader, or even better, again, just use Dapper. Turn these things into actual objects and have Dapper give you back a list or an enumerable of those objects. Down here, we're working with this colorId. I like that it's named colorId here. I mentioned up above that we probably want to change color to something else that indicates that it is, in fact, an ID. It's named properly here, but it's not consistent. So call it colorId everywhere you need it. One thing about this PricingColors table that you need to understand is that it's not just a list of colors. It doesn't just have names and hex values or something like that. This is where we're getting some of that critical pricing data. This is where we're determining whether or not it's a color per square foot, an area-based price or a wholesale price, which depends on the quantity. And all that important logic lives right here, but it's buried so deep inside of this method that it's really hard to find it and really hard to test it. If there's any problems with this logic, it's pretty hard for us to unit test. Moving on, we're going to check the order again and do another INSERT, otherwise we're going to do the PriceReport. This is for features now. That could all be done the same way we've done the other things. Then there's this extra special code here for island logic. We might want to figure out a way to put this into its own method so that it can just check what it needs to do for islands. It's going to add some stuff that has to do with paint. And then if there's paint, it's going to again check the order or check the PriceReport. And so applying non-cabinet special pricing rules for the walls of the kitchen could probably be its own method rather than being a part of this much larger method that we would add. Finally, we're going to output the String.Format logic. That's going to change because we're going to return an actual type, so we won't have to do any of that. If we do need to display it in this particular format for some reason, that can certainly be done as an extension method or a helper on that return type that we're going to create. I already mentioned that I would take the exception handling and put it in its own place. And then this finally block I would not need because if I am going to use a StringWriter here, and I would rename sr

to be sw or something like that for StringWriter, I would certainly put it inside of a using block so that we would not need to explicitly have a finally clause in order to clean it up. All right, so that's what I've got. When I look through this, how many of those did you get and which things did you have that I didn't? If you want to, feel free to leave something in the comments and see what other viewers of this course had to say. Now we're going to finish up and create a plan for how to attack this.

Creating a Plan for Addressing Problems

Once you've identified the issues with some legacy code that you think you want to address, of course, it's helpful to make a plan. For very large refactoring projects like this one, it can be helpful to identify many of the individual tasks that must be completed and track them either as traditional work items or at least as somewhere you can wrap your head around the full scope of the work. You want to find a good starting point and some idea of what done would look like. A simple approach is to just open up a spreadsheet and just add the things you need to do to it row by row, maybe with some additional notes in other columns. We'll talk about what might be there in just a second. Another common approach I use is note cards or sticky notes, possibly using a virtual whiteboarding tool like Miro or Microsoft Whiteboard. And, of course, you can just use your existing issue tracking software, which may make sense in most enterprise environments, but you may want to use a lower fidelity tool first before identifying and fleshing out the things to add to Jira or Azure DevOps or GitHub Issues. Now, something kind of in between that I like about GitHub Issues is this example right here that I just made. You can actually just create a checklist inside of GitHub Issues. And then if you want to move this into something where every one of these is tracked as its own issue with additional metadata and prioritization and all that, you can just go and convert any one of these checklist items to an issue. Once you've decided where you're going to track that backlog of tasks, it's helpful to consider a few things about each one. First, how much work is it? Many automatic refactorings should take close to zero time to perform, but you'll still want to do at least a little testing to ensure that they didn't somehow break something. Other refactoring tasks can be much more involved and can touch many different parts of the system and thus might require more effort, both to do the work and to test that you didn't break anything. And of course, that leads us to risk. Each task should be evaluated based on its risk. How likely is it to break something and how big a deal is it if it does? You can use standard risk management approaches here, considering both the risk likelihood and severity to arrive at a risk

level, which might be something like low, medium or high risk. Finally, how worthwhile is the change? Does it make the code much easier to work with? Does it make it easier to test or to extend in ways that you expect that you'll need to be extending it? Once you have this information, you can prioritize the tasks that have low risk and effort, but provide high value. These are typically referred to as low-hanging fruit. And then, once those low-hanging fruit items are done, it may be time to consider which riskier and more difficult tasks should be tackled next.

Summary

In this module, we examined some complex legacy code. We reviewed the code to gain an understanding of its function, as well as various code smells and principle violations. Then, we documented these opportunities for improvement in some fashion, depending on our needs and the needs of our organization. Finally, we added some metadata to our list of work, covering things such as effort, risk, and value. We use this to prioritize the work so that we could make an initial plan to start addressing the issues. Up next, we'll apply some of the refactorings to the legacy code in order to improve it. The next module is going to be mostly coding, and I hope you're as excited as I am to see how this codebase is going to look once it gets a bit of much needed attention.

Refactoring Legacy Code to Follow SRP

Introduction

Welcome back, friends. I'm Steve Smith. And in this module of Refactoring to SOLID C# Code, we are going to be doing just that, refactoring. We analyzed the code we want to work on in the previous module, and now it's time to try to apply some of the fixes we've determined are likely to have the biggest gains. We're going to start by considering SRP, the single-responsibility principle, and how we can use it to guide some changes that will improve the design of the system. We're going to start with a brief note about characterization tests and why they're important. Then, we're going to dive right into

the work of refactoring the code. I haven't shown the analysis and conversations that would've gone into prioritizing the work. Just assume that that happened and the results are the set of things I'm about to show you. At a high level, we're going to make three sets of changes to the legacy application. In this module, we're going to attack the single-responsibility violation by introducing some more types, which will also reduce our use of statics and the code smell known as static cling. We'll also reduce the use of primitives and make the code follow Liskov's substitution principle better as a result too. In later modules, we'll focus on adding interfaces and supporting dependency in version and then on applying the open-closed principle and Liskov's substitution principle.

Demo: Running the App and Characterization Tests

If you're not familiar with characterization tests, I've discussed them in my other refactoring courses in more detail. Briefly, they are tests that you add to a legacy system in order to ensure your refactoring efforts don't break existing functionality. They are usually end-to-end tests, and they may even involve some manual steps. But the hope is that after a bit of refactoring, you'll be able to write better, smaller scoped integration and unit tests and you won't need the characterization tests so much. Let's run the app and show its output currently and then identify how we can create some basic characterization tests to ensure that we're not breaking things. All right, so now we're going to actually run the app for the first time. I know you might find it weird that we didn't run the app that we're going to use as the basis for this course in the other module, but it's really not that unusual when you're looking at a legacy app for it to be difficult to run, especially locally. And so a lot of times when someone's walking you through legacy code, they're not going to start by running it. They'll just start by diving in and showing you all the code. So, we're going to go ahead and run this, and we're going to hit this Index page right here. And we'll show you what that looks like right now. So, the site does use authentication, and there's a built-in admin user that you should log in as. It's admin@test.com. And then the password is Pass with a capital P @word, all lowercase, and then 1. So, we're going to log in. And then here, we're going to see this Index page. You can see we're logged in, and it's looking at some details of a kitchen. Now it's not much to look at. I built this just for demo purposes, but it does have the expected values here for the wall order. And so here you can see the numbers that we're getting back, and this is what we're going to try and verify is working when we start to refactor the code. You can also use this to generate a file. And so that's going to drop a file, a CSV file, into a

location, or you can place an order. And so this will actually create an order. And so if we look at the code for those, it's the code we saw in the last one where it's going to actually call `CalculatePrice` for `PriceReport`, or for `PlaceOrder`, similarly place the order. And of course, on the Index page, in that case, we are just putting the call right inside of the code so that as we loop through each wall on a given kitchen, it's going to show the calculated price for that wall. Now to do a characterization test, what we're going to do is we're going to go and look at first the file that was created, and that goes in your bin folder under whatever version, right now I'm running .net7, as `Orders.csv`, and it looks like this. So here's the Smiths Kitchen. We just ran it right now, and here's the headers, and here's the individual parts, Okay, there is a tool we're going to use in our characterization test called `ApprovalTests`. And so this `ApprovalTests` package is the one that's going to do the heavy lifting for us. And so in here, what I'm doing is I'm just grabbing the CSV file, which in this case is going to be that `Orders.csv`, and I'm manually copying it into the test's bin folder. We could automate that, but for now it's just a manual step. I'm not expecting to have to do it all that often. The other thing I'm doing is because there's a date stamp inside that file, I'm doing a little bit of work here to strip that off so that we're not going to have the date stamp. And so if we run this test right now, I've already pulled in and showed an approved version of the output. And so you can see this test passes. Now if I wanted to make it fail, there's a few things that we could do. The easiest one would be to just go and find that `Orders.csv` file that it's using as its source here, and we can just modify it. And we'll say the paint, well it actually costs \$4. So we'll do that. We'll save that change. We'll go back to our test, and we'll run this test. And what it's going to do is show us the difference. And so when that runs, it pops up this diff tool, WinMerge in this case, and you can see it's highlighting that the 4 I changed from a 3, and it's letting us know that something has changed, and it's using two different text files. One is the received file, that's the one it just grabbed. That's the new one. And then the approved one is the one that it is expecting. Okay, so that's `ApprovalTests`, and it works great for these types of characterization tests. All right, now let's get into our refactoring.

Demo: Refactoring to Follow SRP

Now we're ready to do the first phase of our refactoring. We've decided to focus on the single-responsibility principle. And to start, we're going to add some types in order to reduce the amount of code and primitive obsession in the `CalculatePrice` method. We're also going to extract

low-level code into separate methods, which we'll work with more in later refactoring steps. All right, so the first refactoring that we're going to do is we're going to change a static to an instance. That should be a fairly straightforward change. Once we've changed it, we're going to lean on the compiler to find all the places that we need to update. In most cases, we'll just be able to new it up in place. However, on this Razor page, we're going to have to introduce a local. And with that change, everything now compiles. Obviously, it probably won't be this simple in a much larger legacy application, but the steps will be the same. With everything compiling, you're going to want to run your characterization tests and probably do a commit to make sure that you're recording each one of your known good states and then continue on with your next little refactoring step. In our case, the next thing we're going to do is look at the primitives that are being passed into CalculatePrice and try and pull that out into a separate type. In this case, we're going to use a record struct so that it will be passed on a stack, and it won't have any additional impact on memory on the heap. And we're going to take those four primitive types, two ints and two strings, and just bundle them together into a PriceRequest. This will make it so it's much more clear, as we'll see in a moment, what variable is being used for which property when we create that PriceRequest as part of calling the CalculatePrice method. An incremental step that we can take is to add an overload to PricingService that uses this new method. This is especially helpful if you have a large codebase and you're trying to incrementally introduce this new change. Here's what that looks like. Now that we have the overload, we just need to go find all the calls to CalculatePrice and change them to use a PriceRequest. We can use the references on CalculatePrice, or we could just make it private and then find everything that breaks. Either one of those will work. In each case, we're going to now create a new PriceRequest and pass that in. The new code looks like this, and you can see that by using the object's initializer syntax, it's very clear what each one of the values is. We're going to go ahead and make the same change everywhere else that we're using the PricingService. At this point, there's only one reference left, which is our overload. We've moved all the other references to use the priceRequest parameter type. So now we can remove the original method signature and only use the priceRequest version. Of course, as soon as we do that, we're going to have to prefix any of our usages of the original parameters with priceRequest dot. (Working) Now that everything builds, we're ready to move on to the next refactoring, and we can move this PriceRequest type into its own file. Now let's look at the return type. CalculatePrice is returning a string, which is an odd choice. And if we look at the very first line of code in the method, we see that it's returning a pipe delimited set of 0s.

Obviously, that's not going to be the type that we really want to return. Since this is returning back numeric pricing data, it should really be using a decimal. However, it does need to return three different values, so we can't just use a decimal as our return type. We actually want to return a record that includes all three decimal values. So we're going to once more introduce a new type. This time we're going to call it a PriceGroup, and it's once again going to be a record type. Now here, we're going to return PriceGroup instead of string, and then we'll need to find all the places where we return a value and return a new instance of PriceGroup in those locations. There aren't that many places to do it. Here, we've taken care of the first one. There are apparently four others. All right, now in this case, we're not actually returning a numeric value. We're returning a string to indicate an error, and the same is true in the next one. So we're going to address those in just a moment. Let's find the last place where we actually return it. That'll be this one right here. We're doing a String.Format on those values. And so instead, we're just going to return a new PriceGroup with those values. And again, we'll let Copilot help us out. It looks like it's giving us the right things except that these are floats. We're going to want to change that, and we may want to override ToString to keep the same behavior. So let's go and do both of those things. First, since I have the String.Format in my clipboard, I'm going to go ahead and add that to PriceGroup. We're going to override ToString, and we'll just paste in the value here. Now that's set. Now we need subtotal, subtotalFlat, and subtotalPlus changed from being float values to being decimal values. Now that's going to require us to change a bunch of other things to decimal as well. So basically, all the different costs or price-related local variables are going to need to be converted from float to decimal, which is what they should've been to begin with. In addition, anywhere that we're trying to read the values from the database, those will need to be decimal as well. We also have a call to GlobalHelpers.Format, which is expecting to take in a float. We're going to need to change that so it also works with decimal, and so we'll just add an overload. All right, once you get through changing everything from float to decimal that needs to be changed, we're left with just the three errors where we're actually not returning a numeric value at all. For this scenario, you have a couple of choices. One is you can raise an exception, and so that is a valid choice. This one, however, seems to be indicating some other type of problem. It's saying there's a session expired as opposed to invalid input or something like that. And the one next to it is invalid kitchenId, which is again some type of validation. Both of these things we could just move to the priceRequest type, and eventually we might do that. But another approach that I like to take is to introduce a result type. And by using a

result type, we can take these different types of results that are not successful or not happy path, and we can encapsulate those in the return type without having to resort to exceptions. Let's go ahead and see what that would look like. For this to work, we're going to need a result type abstraction, and I'm going to use one of my own NuGet packages called Ardalis.Result. Once we've installed this package, it doesn't actually introduce any dependencies, but it does allow us to abstract different types of responses in addition to the success response. So, the first thing we need to do is change our return type to be a result of PriceGroup. We're going to pull in our namespace. Now if we're trying to return a non success result, we can just specify what that is at that point in time. For successful results, you notice we don't have to change anything. We can still just return a new PriceGroup, and it will work as before. But in these instances, we can change to return something else. So in this case, it would be maybe return with an error or even return Forbidden because it's suggesting that the session has expired. So to indicate that a session has expired and they need to log in, we could return a Forbidden just like the HTTP status code of the same name. If we want to show that a value was invalid, then we could just return that result. Now with that change, it's much more clear what the issue is, and we're not trying to overload a string to represent both success and error cases. Finally, there's one more place where we're also specifying that something is invalid. We'll use the same approach there. Okay with that working, now we run our tests again and everything and make sure that we're in good shape. Do another commit. Let's jump back up to the top. We can move PriceGroup to its own file. And now let's continue looking at our instance variables for this method. These two, it turns out, were never actually used, so we can remove them. Now one last thing that we'll have to change is in places where we want to display the value such as on the Index.cshtml page. In order to access the value, we just have to add .Value here. And with that, you can see that everything is still working the way we expect. Next, we're going to look at the different types that are being instantiated inside the method. I don't like that these are being created all at the top and not closest to where they're being used. But also, if you look at them, you can see that there are groups of things that are together. They're proximally similar in terms of how they're listed out in the code, but they're not grouped together as a type. So, things like thisPartWidth, thisPartDepth, thisPartHeight, that could be a type with three properties. Same thing for subtotal, and Flat, and Plus. So, we're going to go ahead and make some of those changes now too. Let's go ahead and add a subtotal record struct. Now because we want to be able to modify the subtotal as we go through the method, we do want to make it mutable. So we're going to give it

properties that have getters and setters, and we're going to replace those three instance variables with one Subtotals value. Now of course, we know that's going to break some things. So we're going to run through the code, find anything that was broken, and modify it to use this new type. Notice that if you're careful in how you name your variables, it can make it so that it's pretty easy to make this change. Now you might notice that PriceGroup and Subtotals both have kind of the same values. They're very similar types, and so we might decide we want to refactor those together at some point. But they have different intents, and one of them is meant to be modified throughout the life of this method, whereas the other one is meant to be a return type that is populated and then shouldn't be mutated after that. So, they don't really necessarily need to share the same type, even though they have essentially the same three properties. All right, with this change, everything compiles, move this to its own file. Now we're going to do basically the same thing with the part. So basically, anything that refers to this part, we're going to put that all into another mutable record that we can work with. We're going to call it Part. We're going to give it all of these properties, and then we're just going to create ThisPart. And now that we have that, we can get rid of all the things that are referencing ThisPart. With that change, we now have 59 errors to go and adjust, but all we have to do in those places is replace thisPartWidth with thisPart.Width. As you can see by naming it ThisPart, it makes it so it's easy to make the changes. We just have to introduce a dot at the appropriate place. And for Color, we don't want to just call it thisPart.Color anymore. We want it to be colorId. And so the property on our new type is colorId since that's an integer ID value and not the actual text of the color name or something like that. Now with that in place, we can just move this to its own file as well. With that done, our full list of local variables is not quite as bad as it was, at least up here at the top. There's a few more declared down below. So, we're going to move on. And the next step is going to be to start pulling out some low-level logic into its own methods. The first one we find is this using block, which is fetching the current wall for a given kitchenId and wallOrderNum. That's going to populate a data table. So, we ought to be able to extract this out to a method that just populates a data table. We're going to continue to do this for each additional query that we see that's using low-level code with an ADO.NET connection and a using block. The next one is CreateNewOrder. Then we come to one that is selecting cabinets for a wall, and this one's going into the dt2 data table. Now if we have a special price for a SKU, we're going to go to the pricing SKU's table and look up the cost for that part. So, let's pull that out into an appropriately named method as well, and then let's do the same thing for this one.

We're going to fetch all of the markup information for color. Now at this point, I notice there's a couple of local variables here that I'm not going to want to pass by reference if I can avoid it. So, I think I can pull those into the thisPart variable. And in fact, I already had properties for those. If we go look at Part, we have ColorMarkup, we have ColorPerSquareFootCost. So let's go ahead and pull those out of their local values and use the ones on thisPart. So this is going to end up being this.ColorMarkup and thisPart.ColorPerSquareFootCost. ColorSquareFoot wasn't a very good name, so ColorPerSquareFootCost I think is more descriptive. Let me just go remove thisColorMarkup and thisColorSquareFoot items here. And thisLinearFootCost is also on Part, so I'm going to pull that one out too, and then we'll fix any of those things that are related. All right, now with that change, we are ready to extract out the method for getting the custom price for the color. And notice because we pulled everything into thisPart type, we're only having to pass in a reference to this part. We're not having to pass in by reference some various local primitive values. Next, we want to get the user markups, then we're going to insert the Order item. Here again, we're going to load a data table. This time, it's dt3. And we have a nice comment here that says get feature cost. Usually when you have comments like those, they're a good tip for what you want to call your method. Now, in this case, notice that it's passing in by reference the part, the subtotal, a bunch of other things. When you see your method has a bunch of things like this being passed in and some are by ref and some aren't, sometimes that's a sign that you haven't really gathered the code the way you want. So let's undo this one. Now let's take a look at what's going on here. So, in this using block, I think I only grabbed that much code. So notice this curly brace goes a lot further, all the way down here. And in fact, there's a few different calls in here. So, we're going to want to break this up a little bit differently. The real problem here is that we're using the same connection object for much longer than we need to. So instead, we can just make a few different using statements, and then this will be much easier to extract. There's no reason for us to be reusing it. So, we'll just block off that using statement there, and then we'll just create another one right after it. We don't need to close it immediately. That wouldn't be helpful. And in fact, we don't even necessarily need it to be right here. It could be somewhere down here where we next need to create a connection. So let's leave that for each out of it. And then we'll create one here, I think. Now at this point, we'll need to create an instance of command, and we're going to open up the connection, call it DataReader. Let's just follow this down. Here, there's another command, command 2 or conn2, inside of there, but this one really doesn't need

to be that long. Pretty sure that once we've loaded up all these data types here, we could pull that one out. So there's the using, using DataReader. I think if we stop the using right there, we'll be fine, and that's going to jump now to this one. And we just need for that to go away. And then this will need to be a var command again. And really, we don't need this to be conn2 anymore, but we'll worry about that later. So, at that point, we've cleaned up the using statements so they aren't circling around each other quite so much, and each one is now its own self-contained call. And so we can now just take this one and try and refactor it to a method. And notice that now this method is much simpler. We're going to do the same thing for PricingColors here. The trick is that this one wants to work with the subtotal inside of the DataReader and inside of the using block when it really doesn't need to. All these values here, featureCost, TotalFeatureCost, UserMarkup, those are all available to us further down in scope. So if we pull these three calls here to subtotal out of this and instead we put them inside the if block but outside the using block, that means that if we take this using statement and extract it out to its own method, it should be clean just like the other ones were. And so in this case, we're just loading out feature, color, cost info, Extract method. And notice again, this method is taking in a lot of different arguments, so that's telling us we probably need to adjust something here. And when we look at this, we can see it's taking in the Part, the colorId, the quantity, the featureHeight, the featureWidth. Basically, there's a bunch of primitives all related to feature. So, what that's telling us is we want to do the same thing that we did for the part with the feature. So here we have a list of all the primitives that are being set for feature. We're going to take this collection of primitive local variables and put them into a single feature data type that we can then use instead of all these locals. All right, so now we have feature, and it has FeatureId, ColorId, ColorName, SKU, Quantity, Height, Width, Price, Cost, MarkedUpCost, etc, etc. We're going to use that instead. We're going to need an instance of it somewhere. We don't need it up at the top. It's going to be too far away from where it's used. Where we need it is just inside this for each loop. And then we just need to convert all of these different assignments to local values to assign to the properties on feature. And then anywhere we are using one of those local variables, we just need to prefix it with thisFeature dot. And then we go back to looking for things to extract out to their own methods. All right, now that was a lot of refactoring. We pulled out all these methods, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 different methods, which we're going to probably put on other objects later on. Last thing I want to show you is the code metrics. If we look closely at the CalculatePrice cyclomatic complexity, it is now down to 20, which is a significant

improvement over what it had before. And we're going to keep on improving that as we continue to refactor.

Review after Refactoring to Follow SRP

Thanks for sticking with me on that. I know that was a lot, and trust me, it took me much longer before I edited it down. With real-world code, it can take days or even weeks to make these kinds of changes. Now that we're done with that initial pass of refactoring, what's changed? Well, we have a little less tight coupling because we're using fewer statics, and our low-level data access code is no longer directly in our method. We also have fewer primitives, both in the method signature, as well as throughout with its use of local variables. And that makes the code easier to understand and also has the additional effect of making it easier to extract methods. And finally, we've lowered the overall complexity of the method. We can tell that just by looking at how long it is. It's down to about 184 lines of code. But also, as we just noted, the cyclomatic complexity has been reduced to just 20, which is not bad, but it's still way more than we would like. We'll keep improving on that.

Summary

In this module, we started improving the design of the legacy code. To ensure we weren't breaking things, we created some characterization tests to preserve the original functionality. Then, we turned our focus to the single-responsibility principle. We identified responsibilities we could pull out into separate methods and sometimes types, and we extracted behavior from the main method. We also reduced coupling by removing static calls and reducing the overuse of primitive types. In the process, we introduced several smaller types with specific responsibilities and uses. The end result is that we have more types, but smaller methods, making our code more manageable. In the next module, we'll pick up where we left off and start applying the dependency inversion principle and interface segregation principle to the application.

Refactoring Legacy Code to Follow DIP and ISP

Introduction

Hey, everyone. It's me, Steve Smith, a.k.a. Ardalis. In this module, we're going to continue on our mission to improve some real-world inspired legacy code using refactoring techniques and SOLID principles. Specifically, we're going to be applying the dependency inversion principle, or DIP, and the interface segregation principle, or ISP. Our top priority in this module is to start adding some abstractions, which will produce looser coupling. When everything is static and buried in a single method, it's very difficult to find seams in the code, which can be used to test or modularize the design. We are going to apply the dependency inversion principle by introducing interfaces, which will also follow the interface segregation principle for certain common behaviors. We'll invert the dependency so that we can inject it into the pricing calculator. Both the original pricing calculator code and the implementation code we are extracting will depend on the interfaces we create rather than directly on one another.

Demo: Refactoring to Follow DIP and ISP

So let's get started. We need interfaces describing certain kinds of behavior that we want to pull out. And we'll need concrete implementations that we'll wire up with the DI container to inject at runtime. Let's start by looking at the methods that we extracted in the previous refactor. These were all of the low-level data access methods, and each one was doing some type of I/O to the SQLite database. The first thing we want to do is see if we can group some of these together so that we can create a new type, a new interface that has a few of these methods on it and therefore is somewhat cohesive. We want to follow the interface segregation principle, which means we want to keep our interface fairly small and focused. And since we know all these methods are already being called by the PricingService CalculatePrice method, we know that any collection of these we grab will technically follow the ISP principle, but it's going to be better if we don't just take all of them and throw them into a service from the start. So, looking at some things that kind of pop out that could be useful to see that

things are related, I'm going to say that things that have an Order parameter are the ones that I want to look at first. So I'm going to create a new interface that has to do with orders. So let's just jump down here where we can type that out, and we're going to create a new interface for order data access. And then we're going to add each one of the methods that we see here that have to do with orders. There are four of them, `CreateNewOrder`, `InsertOrderItemRecord` for Part, `AddOrderItem` for Feature, and `AddOrderItem` for a wall treatment, which is also Part. So let's see what it looks like when we've created that interface. All right, with a little help from Copilot, this is what we've come up with. We may modify this a little bit as we start implementing it, but the next step is to create an implementation of this. And since we're using SQLite by default, we're going to name our implementation accordingly. And then we're going to implement the interface. As you can see right now, everything is not implemented. We're just going to copy/paste the other implementations from our static methods that we extracted into each one of these. Now that we have our interface and our class that implements everything, we're ready to move these things into their own files and then modify our price calculator so it takes in this `IOrderDataService`. Now we're using dependency injection to pass in the `orderDataService`. The next step is to find all the places where we were using those old methods and replace them with calls to `orderDataService`. The easiest way to do that is going to be to delete the methods that we had previously extracted. Once those have been deleted, you can see over here on the right a red box for each one of the errors that has occurred. We're going to go ahead and just prefix those calls with our service that we injected. At this point, we can go ahead and build and verify it still works. We can just new up the SQLite `orderDataService` and pass it in, and that is an option. With that in place, we can now build, and we can also run the application. As you can see, everything is still working. Now let's go ahead and finish this up by making sure `PricingService` itself can be used with DI. For that, we're going to have to add an interface. Technically, we don't have to, but it's better if we have one. Once we have the interface with the `CalculatePrice` method on it, we're going to go ahead and move that to another file, and then we can just go into `Program.cs`. We'll add a new section here for App Services, and we're going to add the `IOrderDataService`, pull in a namespace, and then we're going to add the `PricingService`. We can build, and things should still work. Of course, they do because we haven't actually changed how we're instantiating `PricingService`. So the last thing to do to verify that works is actually inject in a `PricingService` into these three pages. So to do that, we're going to go ahead and just add that type, do the same thing over here, and then here we just need to inject

it. (Working) And you can see everything still works as before. The next thing we're going to look for is some common if logic that keeps repeating itself. We'll see if we can figure out a way to improve that design. So as we scroll through the code, we can see there's a little bit of if logic here doing validation. We'll take a look at that in the next refactoring. And then we can see down here there's a `priceRequest.refType` call. And if we look for `Request.refType`, we're going to see it's used to create a `PriceReport` or a new order. Later on down here, it's used again for an order or a `PriceReport`. Now they're in a different sequence, but that doesn't really matter. Down here, again we're going to add an `Order` item or update the report. And then down here again, we're going to add the wall treatment or add it to the report. So, there should be a way for us to introduce an interface that will allow us to replace this conditional with polymorphism. And by polymorphism, I mean we'll be able to call some instance of this interface, and it will automatically do the right thing, depending on which strategy we have provided, either an order strategy or a pricing report strategy or just a default strategy that performs the pricing updates with the subtotals because we want to do that every time. So let's start with this first one, the first if statement that we found. And this appears to be sort of setting the stage for future operations. In the case of the report file, it's adding the headers and opening up the correct path. In the case of an order, it's creating a new order. So, I think if we have some strategy that we're going to use for this, it would make sense for it to have an initial method that might be called of something like `Create`. And in order for it to do what it needs to do, it looks like in the `PriceReport` case, we need kitchen's name and ID. And then down here, we need an order, but the only thing we know about that order is it needs to have a `kitchenId` on it. If we go look and see where order was last referenced, it's way up here where we said order is a new `Order`. So, I think we can use just kitchen, and that'll be what we need to pass. So, let's create a new interface and put it at the top this time. And we need to give it some kind of a name. How about `IPriceCalculationStrategy`? We're going to be explicit about the fact that it is a strategy, and we're using the strategy design pattern. Now, each one of these methods is probably going to be void because there's really nothing being returned by any of them. They're just performing some operations on a file or on the order. And the first one we said was going to be `Create`, and it needs to pass in `Kitchen kitchen`. Next, we're going to need to implement this in a few different ways. First, we need a class that implements this for a report. For that one, we'll call a `PriceReportCalculationStrategy` that implements that. And then another one called `NewOrderPriceCalculationStrategy`. And then we need a third one to represent when it's just a default

case. Now in the default case for Create, we know it's not going to do anything. And so we'll just add a comment explicitly saying that we mean for this to do nothing, so it's very clear that we didn't just forget. Now the other two, we just need to go grab the appropriate logic and move it into the appropriate place. So, back down here, we're going to go find our PriceReport code. We're going to grab all this code here for the StreamWriter, and we're going to move that into the price report strategy, and the price report strategy is this one here. We'll paste that in. Now we don't have this local variable anymore for the StreamWriter, so we need to create one, and now's a good time to give this a better name. All right, now StreamWriter is going to start out null here with no value, but then we're going to new it up on this line. And the reason we're creating it as a field is because we're going to need to access it in future method calls that we're going to implement. Now because StreamWriter needs to be disposed when we're done, we should implement IDisposable, and that's going to be pretty easy for us to do here. We're just going to say if StreamWriter is not null, we'll go ahead and close and dispose it. Now we have the NewOrderPriceCalculationStrategy still to do. Let's scroll down and see what happened there. We need to create a new order. We'll put that logic right here, and we're going to need an order that we're going to reference throughout the life of this strategy. So, we'll create a new order here. There we go. Now, it turns out that this order strategy is going to actually need that orderDataService that we just injected down here into the PricingService. So we're going to pull that also up here, and then we can assign that from its constructor so it gets populated. Now everything is building. We've created some new types. We know nothing is depending on them yet, so let's change that. Let's make it depend on them. We're going to make a call here to our new order pricing strategy. Of course, we don't have a price calculation strategy yet, so we're going to pass that in as a method argument. And the reason why is because we want to be able to change which strategy we use based on every time we call this CalculatePrice. We don't necessarily want to instantiate it with a particular strategy every time inside our application. All right, so now that looks correct, and we can get rid of these if statements. So, notice we're going to get rid of about 15 lines of code or so here and replace it with just 1 because we've moved that logic into those other services. Now before we go any further, let's go ahead and compile this, and we're going to see that there's a couple changes we're going to need to make elsewhere. So right away, we're going to see PricingService doesn't implement IPricingService, so let's go ahead and fix that. The problem is that we added another parameter to CalculatePrice, so we need to put that on the interface now as well.

Then, more importantly, we have to be able to pass in that strategy each time we want to use this CalculatePrice method. And so we created three of those types, as you recall. And the one that we want on the home page, Index.cshtml, is the default one. Oh, and I see now that we put this in the wrong place. So, this stuff here didn't go in the default one. It should've gone into the `_newOrder` one, so let me just fix that real quick. Now you can see this is all fine. We're able to pass in a new instance of the default strategy. However, inside these other locations, we're going to want to pass in a different strategy. And so to get this to work, we have to insert those strategies into DI and request them here. And the reason for that is mostly because if we look at the strategies again, mostly because the `_newOrder` one requires an `IOrderDataService`. The report one doesn't have any constructor dependencies. And of course, the default one does not either. But since the `_newOrder` one does, the easiest thing for us to do is to just register all of them into DI and then request the appropriate one in each place. It's likely to change later on that we will have dependencies, so that's why it's just better to do it this way from the start. All right, so let's go into Program.cs, and let's add a few more of these registrations. This time, we're not going to assign them to an interface in the case of the non default ones, but we will use the default one as our default for the interface. Now with these types added, two of them with just the class name and one of them with the interface, we can go ahead into those pages and inject in the appropriate types that they need. So, here with the file, we want to pass in a `_priceCalculationStrategy`, which we're going to get from the constructor. Because this is the file generation one, we're going to request the `PriceReportPriceCalculation` strategy, and then we'll just name it the appropriate thing. Here, we can assign it to an interface if we wanted to or the type, either way. I'm going to leave it this way so it's just clear that we're using a concrete type for the generate file approach. Now we're going to do the same thing for `PlaceOrder`, but we're going to use a different variant. And with that, everything should work. And then finally on `Index.cshtml`, we can just inject an `IPriceCalculationStrategy`. And then down here, we can use that price calculation strategy in place of newing one up directly. Once again, we see everything still works on the home page. We can make sure we run our test to verify everything else. Now we're going to return back to `PricingService`, and we're going to go and do the same thing for the other methods. So we have `Create` so far. Let's go ahead and do a search for `refType`. Here's where we are calling `Create` on line 118. The next place we're going to use `refType` is here. We'll just highlight it so we can see all the different instances. And now we're going to add an `OrderItemRecord` or add that rather to the report or do nothing. Then we're

going to add a feature, then we're going to add a wall treatment. So, let's add each one of those necessary methods to our interface. Now we know we're going to need more parameters than these, so let's go look at the first one again where we need the part. If we need to add a part, we need the order, which we're going to have. We need the Part, we need the UserMarkup, and we need the part's MarkedUpCost, which, of course, we can get from the part if we have it. So really just the UserMarkup is the other thing we need. And then for the report, it just needs the Part, the Part, the Part, everything about the Part all the way across. All right, so as long as we have the Part, it's happy. We just need the UserMarkup. So, let's go up here and add part, decimal, userMarkup. And last thing we need to do, a do nothing. Now we need to go ahead and call AddPart at the appropriate place after which we can go ahead and delete this whole if statement, getting rid of another 15 or 13 or so lines of code. We're going to do the same thing for feature. We're going to look at this if check right here and get rid of it, and then we're going to do the same thing for the wall treatment. Give me one sec while I take care of that. All right, now that we've finished that, we're just calling priceCalculationStrategy.AddFeature or AddWallTreatment. We no longer have any if statements running off of refType. We can take our strategies, and we can move them all to their own files. You'll notice our PricingService no longer needs an OrderDataService, so we can pull that out. And since PriceRequest no longer needs to use a refType, let's go ahead and modify that as well. That has the added benefit of eliminating a bunch of magic strings in our code that was specifying what that refType was. So we've replaced conditional with polymorphism, and we've gotten rid of a bunch of magic strings by doing that approach. All right, so let's go back to our list of methods. Let's see if we can pull out one more. We'll do a simple one. Let's do just the UserMarkup. Looking at this one, we see it takes in two parameters, PriceRequest and the UserMarkup. The UserMarkup, it doesn't really do anything except assign here to return, and then it returns it. And the PriceRequest, the only thing we need from that is the username. So, I think we can modify this into a service that just takes in userName and returns back a decimal, so let's create that. And then once we have an interface, we just need a class that implements it. As far as the implementation goes, we're just going to copy/paste it from our static method we pulled out earlier. We're going to modify it slightly so it takes just the username here. We'll have it go ahead and start with a decimal result, and we'll use that here, and then we'll return it, and that gives us what looks like correct code. Let's go ahead and pull these into their own files. And then very quickly, we're going to inject this into our PricingService. That's why I left the constructor here. We're going to delete our

extracted method. That's going to tell us then where it was used. And where it was used, we're going to use our get. We're going to use our service. We're going to pass in the userName and nothing else. Now for this to work, we need to wire it up in Program.cs. And once that's in place, we should be able to run the application again, and once more, everything still works.

Review after Refactoring to Follow DIP and ISP

Okay, we've made some more good progress on the original code. This time, we focused on adding interfaces and supporting dependency injection. We added four interfaces, two for data access for orders and user markups, one for the pricing service itself, and one for the price calculation strategy, which had the biggest impact on the size and complexity of the CalculatePrice method. Make sure you understand how we did that and what the before and after code looks like so you can recognize similar symptoms in your own codebase. The result of these changes is a much more flexible design. At this point, we can easily extend or replace the functionality of the components we've refactored. Replacing the database or how we're accessing it is simple to achieve at this point by adding more classes. If we have additional ways in which we might need to leverage the overall price calculator process, we can simply create more strategies. We can even apply the decorator pattern to add additional features like caching to things like the data access interfaces that we've pulled out. An additional benefit of these changes is that we've dramatically improved the testability of the software. Each of the individual strategies can now be tested in isolation, and the original calculate price code is closer to being unit-testable. We'll focus more on tests and testability in a later module, but we definitely made some good progress on that front here today.

Summary

That brings us to the end of this module. Just to review, here are the activities we focused on as we refactored with DIP and ISP in mind. First, we needed some abstractions, so we identified and extracted interfaces for common behaviors. Then, we moved the behavior from its original location to new classes that implement our new interfaces. Next, we used dependency injection, ideally through the constructor, but you may also need to use method or property injection at times. Finally, we configured DI in Program.cs for the application so that the built-in service collection is able to provide

appropriately constructed instances to the UI elements that request them. In the next module, we're going to be refactoring one more time, this time to follow the open-closed principle, or OCP, and the Liskov substitution principle, or LSP.

Refactoring Legacy Code to Follow OCP and LSP

Introduction

Welcome back. It's me, Steve Smith, a.k.a. Ardalís. In this module, we're returning to the scene of the crime, this time with the intent of applying the open-closed principle, or OCP, as well as the Liskov substitution principle, LSP, to how we refactor some legacy code. This will be a somewhat shorter module than the previous two. We're going to start by identifying cross-cutting concerns and looking for patterns we can apply that will let the code follow OCP for certain vectors of change. We'll also keep an eye out for misuses of types, including null, and we'll try to ensure our subtypes are always substitutable for their base types, including interfaces. In this way, we'll ensure the code follows Liskov's substitution principle.

Understanding Cross-cutting Concerns

Before we go hunting for cross-cutting concerns, let's spend a moment to agree on what exactly they are. According to Wikipedia, the source of all truth, cross-cutting concerns are aspects of a program that affect several modules without the possibility of being encapsulated in any one of them. So what are some examples of cross-cutting concerns? Business rules are often considered to be their own concern. In our example, things like the structure of the price calculator and the steps it goes through, as well as how it tracks several different variations of base price and one or more markups that are applied, these are all examples of business rules in our system. Persistence and data access are another concern, which often is spread throughout an application's code like cancer. Keeping persistence concerns separate will often have a huge positive impact on the maintainability of an

application. Validation logic and when and how to apply it consistently is another common cross-cutting concern, and logging, at least to some extent, is a cross-cutting concern as well that can be pulled out so that it's not cluttering other code. Some logging usually has to exist within other modules, but things like when something starts and when something ends can typically be pulled out. Likewise, exception handling can often add up to a lot of duplicate redundant code when it could've been applied in a common way as a standard policy. And monitoring, both for uptime, as well as performance and security is another big cross-cutting concern that modern systems should typically include in as modular a fashion as possible.

Demo: Refactoring to Follow OCP and LSP

For this refactoring, we're going to focus on the open-closed principle. We want to be able to insulate the stable parts of the application from less stable parts and vice versa. We're also going to continue to apply the Liskov substitution principle, ensuring that as we create new classes and interfaces, they are completely substitutable for their base versions. And we will keep on cleaning up the `CalculatePrice` method. It's still much longer than we would like, and it has a lot of poorly named or poorly located variables. So let's return back to the `PricingService`, and let's take a look at our `CalculatePrice` method here. The next thing that we want to do is pull out some of the cross-cutting concerns that are in this method. Specifically, if we scroll down into it a little bit, you'll see that we are accessing the session basically every time, so we'd like to pull those things together and maybe extract them. We're also doing some try catch logic that essentially wraps the entire method and also includes some cleanup. Now eventually, I'd like this cleanup to go away and just implement `IDisposable`, but for now let's just move it so that we don't have as much stuff in this type. So when you want to add some functionality to an existing class, there's a pattern for that called the decorator pattern. And in this case, we want to kind of pull out some functionality into its own class so it's not cluttering up this one. And what that's going to do is help us achieve the open-closed principle because we'll be able to apply certain things like error handling to this class without having to actually change the class itself. So we are going to change it now to put into place this pattern. But then thereafter, we won't need to, and that's the beauty of the decorator pattern and how it applies the open-closed principle. So initially, we're just going to create a new instance of `PricingService` that implements the `IPricingService` interface. And to start, it has a `NotImplementedException`. We are

going to pass in the thing that we're wrapping, the PricingService, to the constructor because we're going to want to call that. So initially for our decorator, all we're going to do is pass in the actual PricingService and then just delegate to it inside of the CalculatePrice method, returning whatever it does. Now to finish the refactoring, all we need to do is pull out the pieces that we'd like to have inside the decorator so that they don't need to be inside of our main class implementation. (Working) All right, so with that change, we've pulled out the two references to Session and the try catch logic, including the finally block. And now all of the error handling and session access is in the decorator along with this finally block, which actually I think we can get rid of. Let's take a look. So, in the original method, we have StreamWriter still being declared, but it's no longer used anywhere in this because of some previous refactorings we did. So we can delete it from the original class. And since it's gone there, we no longer need the finally block, and now we should be in good shape that we could build. All right, so we would test this application. And right now all that would happen if we tested it because we're not quite done is that we wouldn't have the session access or the error handling logic in the actual PricingService anymore. There's one more change we need to make, and that is we're going to go to Program.cs, and we have to say when something is asking for IPricingService, we want it to get PricingServiceDecorator now. Of course, in order to create the decorator, it's going to have to have the actual PricingService, which since we changed that is no longer listed, so we need to add that as well. So with that change, moving PricingService to just add the class itself and then having the decorator implement the interface, that means anything that asks for the interface is now going to get the decorator. If we go look at our pages just to remember how this is working, we'll see that this is looking for an IPricingService, so it will get the decorated version at this point. All right, now that we have the decorator working, we can go ahead and move it to another file. And now our existing CalculatePrice method just got that much shorter, that much simpler to work with. So, we're looking at right now total lines of code of just over 100. So the next thing that I want to do is take a look at some of the other local variables and see if we can find some that aren't being used anymore at all. So, the next one that comes to mind is the Order class. An easy way to see if this is being used is we can just comment it out or delete it and then look to see if any errors occur. There aren't any, so we can just delete it, but we can continue to do this with some of the other ones. We can comment it out, and it looks like something right here is expecting it, but it just gets assigned to 0. So if we comment that out, then there's nothing. So we can remove this variable entirely. And as we do each one of these changes,

we're going to want to run our tests and make sure that on whatever basis we're comfortable with go ahead and add another git commit to make sure that we're incrementally making our progress. And if something goes wrong, if we get into a bad state, we can always fall back to the most recent version that we know was good, and we shouldn't have lost too much work. All right, so we've gotten rid of Order, SectionWidth, and previously we already got rid of the StreamWriter. And you can still see looking at these remaining variables that quite a few of them are not really used a whole lot. We're going to see what we can do to get rid of some more of them or at least group them together closer to where they're used. I'd really like to get these data tables out of here. And if I can't do that initially, what I'm going to do is at least give them better names. So let's look at dt and see where it's used. If we scroll down here, we see that dt is assigned from PopulateWalls. It's the PopulateWallsDataTable. If we're going to call this thing the wallsDataTable, why don't we call it that? So, we're going to come up here, and we're going to rename this to wallsDataTable. And we don't need to say DataTable this many times on here, but we can just say right here we'll new it up. That actually ends up getting thrown away anyway because it's going to get overwritten down here. And since that's the case and since we only need it right down there, why don't we declare it at the same place where we assign it? And so we can get rid of that line from up here and instead just do it down here. So now there's our wallsDataTable. What about dt2? Well, if we go find dt2, it's right here. Its first reference is LoadCabinetsDataTable. We can go and maybe change this so it returns this result and then call that a cabinetsDataTable. So let's Go To Implementation, and we see this is void. Let's make it a DataTable, and there's really no reason for it to take in this DataTable. Let's just have a DataTable here for it to use, var dt = new DataTable. And I'm fine to call it dt locally in this little method because it's only being used here for this one thing. But then when we're done and we're out of the using statement, we can just return it. And that's a little better. But now that LoadCabinets call right here needs to have a DataTable called, I would say, cabinetsDataTable. But before we do that, let's rename dt2 to cabinetsDataTable, and that should hopefully get all the dt2 references, except for that one. So now we just want to say cabinetsDataTable = LoadCabinetsDataTable, and everything should be good there, which means I don't need this one up here anymore. All right, and then let's just build. Make sure that's still good. Run our tests. They're all good. Now we're looking at dt3, and what is dt3? I think it's features. So, dt3 is calling LoadFeatures. It has a featureRow in dt3. So, I think we could rename dt3 to be feature or featuresDataTable or featuresTable. Let's do featuresDataTable, and then we can

just take this, and we can assign it right where it's first used right there, and then we can get rid of that one. So now, looking up here at our long list of local variables, it's not nearly as long as it used to be. It's getting to be a little bit smaller. All right, so looking back up here at the top, we've got kitchen being assigned here, but it's not actually used until down here where we call this `GetCustomerKitchen` method. And if we peek at that definition, you can see that this is on `Kitchen.cs`, and it's assigning this property, and then it's making a SQL call in this entity, which I don't love. But then inside of there, if it finds this data row, it's going to set these other four properties. So, what I would prefer is to have our data access be consistent. Right now, we have some things that just go call some random method and return back a data structure. And we have other things where it's actually inside of the data structure, it's going and fetching its data. Let's make it so that this one is being fetched by a custom method also so it's consistent. To do that, let's create another service for that. We have currently a `getUserMarkupService`. We want some kind of kitchen data service or or something like that. So let's create an interface, and then we need to do something like `getKitchenByIdAndCustomer`. So it's going to return a `Kitchen`. Now we can just implement this, and it's going to be a `SQLite` implementation. Good idea to always use the how, the implementation details of your classes that implement an abstraction. So, in this case, since I know I'm using `SQLite`, I'll use the `Sqlite` thing as the name. Maybe if I was using `Entity Framework` or I was using `Dapper`, I was using `SQL Server` or whatever it might be, include that in the name so that the implementation detail is there and you know which data service implementation you have. It's an `IKitchenDataService`, so we can implement the interface. And right now, this class does not follow the Liskov substitution principle because if I plug it in as is, anything that's expecting a call to this interface should expect that this method will be implemented correctly. And at runtime, this is just going to blow up. So by definition, it doesn't follow Liskov substitution. So now we need to go out to `Kitchen` and grab this code, our code here, and come back into our service here that we're creating, and we need a kitchen that we're going to assign things to. So `var kitchen = new Kitchen`, and then `this.kitchen.KitchenId` is going to be that, and all these properties down here are also going to be `kitchen.dot`. And of course, when we're done, we want to return `kitchen`. Now with that in place, we just need to inject that in. Remember if you type just the capital letters, Visual Studio will jump to that type for you. So, it's a good tip if you're using this type of camel casing or whatever type of casing style you're using. If you know that you have certain capital letters you can look for when you're doing autocomplete, just type those so even if you have really

long type names, it makes it super easy to find them. And we're going to assign this to a field, and then we just need to change our kitchen code down below to use this kitchenDataService. So we don't need kitchen here, get rid of that, but we do need it on this line. And so we can say Kitchen kitchen = _kitchenDataService dot. All right, so it looks like that. I'm wrapping the lines just because the font's big so you can see it, but normally this will all fit on one line. And now we're going to use that kitchenDataService. Now if we run it right now, you can expect it's not going to work because we haven't wired that up yet in DI. So we're going to finally go into Program.cs and just wire that up. Everything builds. We could run it, we could test it. It should still work. Last thing after this refactoring, we can go ahead and move these to their own files. And now we're back to just the PricingService again, and it's, again, slightly smaller. Just the CalculatePrice method, still just over 100 lines of code. All right, we've got a part being assigned up here. It probably doesn't need to be. Let's see, where's the first place that thisPart gets used? Way down here. So thisPart is apparently a cabinet row, and it's having all these things assigned. There's no reason why thisPart couldn't be set right here, declared right above that. So we'll go ahead and just pull that out of there, and we'll put it right here. That should do the same, but make it easier to find. Now, if we look at some of the last of these locals, we've got isIsland, wallId, and wallHeight all in a group here. Now if we go find out where those locals are being used, you can see they're all right here. Right after we've created the pricing strategy, we go and we fetch these things from the wallsDataTable. Seems to me that we could use some type of an abstraction or custom data type to hold all the wall data. So, let's do that. Let's just take those four data types and put them inside of a container, and we'll call it wall. So jumping back up here, how I like to just start things in the same class, we're going to create this class, Wall. We're going to give it the defaultColor was another one that was being used, and these three things here, but we want these to be public properties on here. So, after turning those into properties and fixing their casing, this is the type that we have. Now we can just go and replace all the uses of those locals with one use of this new type. So as soon as we comment these out, we're going to see everywhere that they're being used. We'll come down here and we see it's this block of four here. And then also down here, we're going to use it again. So, these are all scoped at the full level of the method, not inside of a loop or anything like that. So we can actually just instantiate our wall right here. So now we have a new variable called thisWall. We assign it from the data table, and then down below, we can make sure to use it as well. So basically, we just need to prefix all these calls to thisWall dot. Once we're done with

that, we can build again. Everything is good, and we can move the Wall class to its own file now and clean up the last of those locals. All right, so there's not a whole lot left here. We've got subtotal, bbHeight, bbDepth, and thisUserMarkup. BbHeight and bbDepth we never actually use for anything. I think it was always just some dysfunctional code that didn't need to be there. So it turns out we can just finally delete those. And if we really need them, we'll come back and find them in source control, but delete it, run your tests, check in, say what you did so that you remove those things and move on with your life. Don't try and keep them there commented out forever. They're not doing any good. We just determined they're not doing any good. Let's make our lives in the future easier and get rid of them. All right, so now we're going to further refactor on this data access stuff because we still have a lot of duplicate code, and we're doing some work here where we go fetch a data table and then we immediately turn around and load a wall from that data table. Wouldn't it be nice if we actually had a way to just do all that in one step? And so we're already using this data service sort of pattern for kitchenDataService. Let's do the same thing for wall. So I'm going to have this GetWall method. I'm going to return a Result<Wall> this time just because there are some other possible things that might happen when you go to return the wall. Now we're going to create our implementation for this. It's going to be a SQLite one again, and we've got our method. Now the next thing we want to do is basically all the code that it's showing us right here, we want to pull in that code to get the wall. So if we scroll down here and find where we fetch a wall, it's right here, PopulateWallsDataTable with a priceRequest. Let's go find this definition and see what it's using. It needs a kitchenId and a wallOrderNum. That's why I pulled those out. So, and as a matter of fact, we can just take this whole method. Just take this whole private static method, and we're going to cut it from here just to save a step, and we're going to paste it inside of here so that we can just use it directly. So, now this, we can say var wallTable =, and it wants a PriceRequest, but we don't need it to be a PriceRequest. So let's just modify our private method here to use the same arguments that we're expecting now. And that's easily done. Okay, so now we want to return a wall. So we need some code to turn this data table into a wall. Well, it turns out we have that down here. Right after we populate the data table, we do some validation, and then we instantiate the wall. So let's just grab all that code, and we'll put it right here except for that strategy stuff. We don't need that. But we do have something here where we're returning a Result.Invalid. And so we want to keep that. So, all right. So we're going to keep that so it returns invalid. And then in here, I guess I shouldn't have named this something different, change all

those names. So now we have `thisWall`, and we want to return `thisWall`, and so now that works. So then, there's two things we have left to do. We need to make sure that we deal with this invalid case inside of our code that's going to call this and that we get the wall when it's successful. So we need to inject now this `wallDataService`, and we need to use that to populate our wall, but first we're going to get a result. Now that we have a result, we can check to see if it is invalid. And if it is, we're going to just return an invalid result. Now in this case, notice that we're going to pass up the `wallResult.ValidationErrors` so that whatever we got that was invalid from that method is going to get passed along to whoever's calling this method. Now we don't need any of these other calls here. We already have a wall, so we can get rid of the call to populate it. We can get rid of the validation check. We'll keep the price strategy. We can say that `thisWall = wallResult.Value`, and we'll be in good shape. We don't need any of this. Okay, the only other thing that's left to do, of course, is to wire it up in DI. And now that's done, and everything is working again. Come back here. We can remove the interface to its own file, and we can move the class to its own file. What's left at this point? Well, we can keep on doing this type of thing. We can do the same thing for cabinets, we can do the same thing for features and wall treatments. And what that should do is start to get rid of a lot of these calls to data rows and mapping things here to a particular part. Let's put all of that mapping logic, all of that fetch something from a data row and instead put that inside the data access method so that we aren't having to deal with it inside this code. What that's going to do is it's going to make it so that if that ever needs to change, `CalculatePrice` is closed for that kind of modification. We can change how we do that data access. We can use Entity Framework or Dapper or whatever else we want to do that's a little more modern, and we don't have to change this code. So this code becomes freed from that type of problem. The other thing that having those things behind interfaces does, recall, is we can add decorators on top of that. So if, for example, we're very frequently having to look up all of the cabinets or all the prices or all of the whatever it might be for a particular type of data, we can catch that, and we can have easy ability to add a caching decorator on top of that data access code without us having to add that code to this method, which is already long enough. Okay, so we do have a little bit of homework left to do, replacing those other methods and pulling stuff out like you've already seen. I don't want to bore you with seeing that same type of repetitive refactoring. Notice that even before I get to that additional refactoring, the cyclomatic complexity now is down to 10, which is kind of at the high end still. That's pretty much the limit of what you want to have. You want it to be 10 or less, but

way, way better than it was when we started and much easier for us to work with. So, I'm going to go ahead and finish up this refactoring. You'll find it in the code samples associated with this course, but I'm not going to show it on the recording. Let's go back, and let's review what we've done with these changes.

Review after Refactoring to Follow OCP and LSP

Now let's take a look at what the code looks like after the refactorings that we've just completed. With this set of refactorings, the original class and its main method are really starting to get under control. We've pulled the error handling out into a decorator, simplifying the underlying method. I also pulled out the validation code as well in the parts where I didn't show the refactoring, and we pulled out session access, which if we want we could always put into a service at some point to reduce the static coupling to session. We continued the long task of abstracting out data access. In a prior refactoring, we extracted the methods. But in this one, we took it further, improving the methods to return strongly-typed results instead of data tables and then moving those methods into separate cohesive services and injecting those as constructor dependencies. At the end, the main focus of the `CalculatePrice` method is on business logic. It's no longer trying to do nearly as many different tasks, and the level of abstraction at which it's working is consistent. When it needs to work with infrastructure concerns like databases, it has interfaces it uses that describe what it needs without being concerned with or coupled to how those needs are fulfilled. Remember, when we started, there weren't any interfaces at all. Now you can see we've accumulated quite a few. I've found that a very useful and common refactoring involves extracting interfaces, especially for static methods or in order to facilitate dependency injection or both. In general, code that follows SOLID principles tends to have many smaller focused classes with interfaces instead of fewer longer classes that do many different things. Now it is possible to go too far and to create nothing but single method classes with tiny one-line methods. At that point, you're essentially doing functional programming, and you might want to reconsider using an object-oriented language like C#. Your goal is to have small classes that are easy to understand, easy to test, and easy to swap out. And remember that your interfaces should focus on what, and your implementation classes should focus on how.

Summary

In this module, we started improving the design of the legacy code. To ensure we weren't breaking things, we created some characterization tests to preserve the original functionality. Then, we turned our focus to the single-responsibility principle. We identified responsibilities we could pull out into separate methods and sometimes types, and we extracted behavior from the main method. We also reduced coupling by removing static calls and reducing the overuse of primitive types. In the process, we introduced several smaller types with specific responsibilities and uses. The end result is that we have more types, but smaller methods, making our code more manageable. In the next module, we'll pick up where we left off and start applying the dependency inversion principle and interface segregation principle to the application.

Assessing and Testing SOLID Code

Introduction

Hello again, and welcome back to this final module on Refactoring to SOLID C# Code. It's still me, Ardalís, a.k.a. Steve Smith, and this time, we're going to review the refactored application. We'll start with a quick tour of the current version of the code, followed by some analysis. Ask yourself as we do this, which codebase would you rather work on, the original one or the one we've modified and why? Next, we'll look at how we are now able to write some unit tests for some of the functionality, and the original code didn't really support unit testing. Then, we'll consider some integration tests as well, which will interact with our code, as well as out-of-process dependencies. And finally, we'll talk about next steps for this code and for you.

Demo: Assessing Your Refactored Application

Let's get started with assessing the quality of the code as we've made updates to it during the course. We're going to look at maintainability, complexity, and the overall size of the code. Remember when we began, this solution only had a few classes, a handful of Razor Pages, which we haven't really

touched, and really just one service that we've been dealing with, this PricingService. You may have also noticed there is a Data folder, and it has some EF Core stuff in there. That's just what I used to set up the initial data structure and seed it. It's something that for all intents and purposes doesn't really exist in this legacy application, so just ignore it. Now if we look at our original code here, you can see that it's pretty long. It's not the longest code you've probably ever seen, but it's over 300 lines of code and has a whole lot of if checks, and code duplication, and mixing of different abstraction levels and things like that. So, we've been working pretty hard to try and clean this up. After the previous module, this is what we've come up with. We now have a number of different services that we're injecting in. I would argue probably too many for this service, but there are seven abstractions, seven different services here that we're passing into PricingService. And then within here, we've got line 37 is where we're starting to call into this method, and it only goes up to about 107. So, about 70 lines of code, maybe. We've pulled some of that stuff out into a decorator. So there's a little bit of responsibility that's been moved here. And you can see we have a lot of these different DataService classes that we've created to implement all those different low-level database calls. Now the other thing we did when you weren't looking at the end of the last module when I didn't show you in the recording is I pulled out the interfaces into their own folder, which you'll find here. And I pulled the models that we created, mostly DTOs at this point, but we could enhance them with additional behavior, into a Models folder. Now in a real application, I would probably have different projects as well. But for now, this just kept the Services folder from getting so big with a bunch of stuff that weren't actually services. That's a general look at what the code looks like at this point. Let's look at what the metrics tell us. You've seen me run Calculate Code Metrics a couple of times in Visual Studio. You can access that from Analyze, Calculate Code Metrics, For Solution, and then you'll get this chart here. Where it becomes really valuable though is when you export it to Excel. Once you export it to Excel, you can start to really sort this data in ways that you can't in this section here. For example, what I like to do is look at the cyclomatic complexity for every different method, and there's no easy way to sort by method or filter out just the methods in Visual Studio directly. I've gone ahead, and I've pulled out a few metrics that I care about, and I've broken them down by each of the different modules that we've done in this course. So here is a chart of Microsoft's maintainability index for the CalculatePrice method. And you can see that when we started it was below 15, and then now that we're done after module 5, we're over 40. This is out of 100. So there's still a lot of room for improvement, but you can see that we've

definitely improved the maintainability index. Now let's take a look at the cyclomatic complexity. This is the cyclomatic complexity of just the `CalculatePrice` method, and you can see it started at 40. And after module 5's refactoring, we've gotten it all the way under 10. Now that green line is our target. We want all of our methods to be under a 10 of cyclomatic complexity. And in fact, we can calculate an aggregate complexity number by looking at all our methods that have a 10 or greater and dividing them by 10 and dropping all the fractions to get a sum for a project or for an assembly that tells us what its aggregate complexity is, basically a count of all the methods that need to be fixed. Next, let's look at the lines of code. This is the lines of source code that is in `CalculatePrice`. And you can see it was almost 400 when we started, and now it's less than 100, about 70, 75. And then the total lines of code for the entire project, remember we've created a bunch of interfaces, we've extracted a bunch of services. That total line of code has gone up a little bit, about 7% total. If we look at the raw data, you can see that the aggregate complexity has gone from a 4, which is not terribly bad, down to 0 after module 5. And that's because the only method with a complexity of 10 or more to start with was `CalculatePrice`. So as soon as we brought it under 10, the aggregate complexity dropped to 0. You can see that we added a few percentage points worth of total project lines of code with each of these refactorings going from a total of about 6800 at the start to about 7300 when we were finished. I think we could probably save most of that difference if we were to move away from using raw ADO.NET code and use something like Entity Framework or Dapper to reduce the amount of repetitive code that we're doing in all of those data access methods. Hopefully this gives you a little bit of an idea of how the code has changed from a metrics perspective. We're going to go ahead and look at how we can actually write some tests now for this code, which would've been much more difficult before we got started.

The Importance of Unit Testability

Let's talk for a moment about why testability and in particular unit testability is important. Let's consider some typical characteristics of code that is easily unit tested, whether any tests exist or not. Such code tends to be loosely coupled. It generally will minimize side effects. Usually, it avoids global state changes and especially use of statics or singletons that impact state. Ideally, there are ways to configure state such as providing abstractions that allow populating data access calls with test data without having to set up separate test infrastructure. Unit testable code is observable. Tests should be

able to inspect the system and observe whether code behaved appropriately or not. Unit testable code has many seams, whereas non unit-testable code has none. And because of all of the above, unit-testable code tends to be modular, supporting the replacement of components with alternative implementations. And perhaps most importantly, unit-testable code has clear, predictable behavior, making it easy to write tests and easy to identify what has gone wrong when your tests fail. While I won't go so far as to declare that all unit-testable code is high quality code, it's certainly safe to say that it shares many characteristics with high quality code. Could a developer write poor code that is unit testable? I have no doubt. In fact, I think software developers are capable of writing lousy code using any language, framework or platform. But as a general heuristic, unit-testable code tends to be better than code that performs the same work, but is not unit testable.

Demo: Unit Testing SOLID Code

Now, let's take a look at some unit tests for the code that we've refactored so far during this course. I've added a `UnitTests` project to the solution and a number of different unit tests. Let's take a look at the `PriceGroup_ToString` method. All we expect from this method is to format three different decimal values in a certain way. And the only things that could really throw us off are off-by-one errors, maybe some additional spaces, whether or not there's commas expected inside of the numeric output, etc. So, here we have a `Theory`, which in `xUnit` lets us run multiple different test cases in one method. And for each one of these three different lines, it's going to run a separate test. I've already run the test, and you can see that all three have passed. If we look at the system under test, in this case, `PriceGroup`, you can see that we are simply taking in three values to this record. And then on the overridden `ToString`, we're doing a `string.Format`, formatting them as currency and a pipe separator between each of the three values. So that's a pretty easy one. Let's look at something a little more complicated. When we create a new order, we want to make sure that that order assigns the kitchen ID because that's part of the logic that's inside of there. Again, this is fairly trivial, but we want to ensure that nothing that we do with our refactoring would break this. So, we're going to create a `NewOrderPriceCalculationStrategy`. We're going to pass it in an `orderDataService` and a kitchen object with a certain `testKitchenId`. Then we're going to create the order, and we're going to expect when we're done that the order that we capture on line 23 is not null and that the `kitchenId` set on that `capturedOrder` should be `testKitchenId`. Now if we want to see the code that's actually doing this, we

just need to take a look at the system under test, which in this case is `NewOrderPriceCalculationStrategy`. We're calling the `Create` method, and the important line of code that we're making sure works is this one right here. If I were to accidentally comment this out and then rerun this test, you can see that the test now fails. If I put this code back in and we run the test once more, now it passes. It's important that your tests fail when they're supposed to fail, otherwise they aren't providing you any value. We have a similar set of tests for the price report calculation. In this case, we want to make sure that when we create the new report that it has the expected headers at the top of the field. Inside the original system under test, this was hard coded. Always write to a file, and we can write an integration test that will check whether or not it was writing to a file correctly. But in this case, we're doing a unit test where we are only dealing with in-memory values. So, we're going to pass in a `StringWriter`, replace the default `StreamWriter` with the `StringWriter` and then make sure that the result that we get from that writer includes the headers that we expect. Now I did make a couple of small changes to this to make it more testable. One of those is that I changed the `StreamWriter` to be a `TextWriter`, which is equivalent from a type perspective for what our needs were. But you can see the names don't quite match now, so we'd probably want to rename that to be a `TextWriter` everywhere in this class. And then I also added these two methods. First, when it configures the `StreamWriter`, which before was just being created right inside of the `Create` method, now if it's been set already, it's going to not create the `StreamWriter` and not create the file. Then I have a public method that lets me replace the default `StreamWriter`, and that's what my test is using to insert in that `StringWriter` that it wants to use to be able to observe the results. There are some other ways that you could achieve something similar. You might use reflection, or you might pass in a dependency for how it does the writing, which would be a better way to do it in the future. But since we haven't made that refactoring yet, this little thing is a little bit of a hack, but it does make it so more unit testable. Now remember, we pulled out some functionality into the decorator, and specifically, we did some checks with the session access, and we also did some validation checks that might've returned invalid results or forbidden results. This first test just verifies that if the session value has pricing turned off, that we just automatically return a set of 0 `PriceGroup`. And you can see that this one works. We had to use `NSubstitute` to create a substitute version of some of the dependencies here, specifically the `IPriceCalculationStrategy`. And then we also needed to do a little bit of work to set up the session store that's being done inside the constructor. With all that in place, we're then able to set

the Session value that we want inside our test on line 27, and then everything works as you would expect. Now for any other tests where we want it to actually do something, we need to make sure that PricingOff is not set to Y, and so you'll see in the next test here, we're setting it to N for no. And then we're going to go ahead and check that we get a ResultStatus.Forbidden if the wallOrderNum is 0. And so that's the name of our test is that it should ReturnsInvalidGivenWallOrderNumZero. Really, that should be ReturnsForbidden. Let's change that right now, ReturnsForbiddenGivenWallOrderNumZero. That's better. So we can refactor our tests when they're not quite right just like we can our system under test, and this one passes as well. Now that I've changed the name, let's make sure that didn't break it somehow, and it's still good. And then we also did want to say that it would return Forbidden if the kitchenId was 0. And so this test looks just like the other one, but now we're checking that the kitchenId is 0, and we're expecting that the validation error will be exactly invalid kitchenId with a lowercase invalid. You can see that one passes as well. And then there's a few helper methods here that are used in the test to create some of the objects that we need. Lastly, we can do much bigger types of tests like the PricingService itself. Now, PricingService is still a fairly large and complicated method when we're talking about its CalculatePrice method. And so setting up this test requires a significant amount of work. Because there are seven different dependencies, we have to mock out a lot of those dependencies in order to run any type of test. I have two tests for this. The first one is going to do another validation check, and that's inside of PricingService. Let's take a look at it. Right at the top, after we check to see if we can get a wallResult, we're going to return a result status of Result.Invalid with whatever the validation errors that we got back from our wallDataService, so we just want these to bubble up. And so I have a test that does exactly that, and it ensures that the wallDataService does return an invalid wall. And when that's the case, we expect there to be validation errors on our call to PricingService that include the invalid wall error, which you can see here. Now if we want to test a so-called happy path where we actually did get pricing data back from our PricingService, that means that everything has to be set up in order to work properly. And so for this example, I'm going to expect that I have a flatCost of 10. There is a colorMarkup value set somewhere of 20, 20%, and a userMarkup set somewhere of 50 for 50%. And with those in place, I'm expecting to get back a result of 3 different values of prices being 12, which is a 20% markup, on 10, which is the flat cost, and then 18, which is a 50% markup on the 12. So to set this up, I need to get the userMarkup from the userMarkupService, and that's going to return the userMarkup of 50. I need to

get a wall that I can use to actually iterate over its cabinets and find the price on those cabinets. Once I have a wall, I can have a `cabinetDataService`, and it's going to return back a single cabinet. That cabinet is going to have a Height of 36, which is important because that's also going to be the exact wall height, so I don't have to worry about anything left over for wall treatments. The cabinet fills up the whole wall. It has to have a SKU in order for some of the logic to run to calculate the subtotal. Its base Cost is the `flatCost`, and its markup I'm just going to set right here to be the `ColorMarkup`. Now, that's a little bit of a hack because in the actual system under test, that's happening in this call to `IPartDataCostService`. But because this thing, this `IPartCostDataService`, is only going to set `ColorMarkup`, I can skip that step and just set it up front for the purpose of the test. So, just realize that it's not a perfect test in that case because I'm kind of tricking it in this case. Then I need to get my list of features. This just has to return a new list. But if I don't call this, then it returns null, and things will blow up. Finally, I'm going to create the `PricingService` with all of its dependencies mocked up accordingly. Some of them can just be basic substitutions that don't do anything in particular, and some of them have specific things that they need to return. But when I'm done, I'm expecting that I'm going to get back a result status of `Ok` and that the value of that result is going to be equivalent to a `PriceGroup` with a 12, a 10, and an 18. And the only other code to show here is just another helper method to get a valid price request. This actually covers a fair bit of `CalculatePrice` at this point in terms of test coverage. The one significant place that I haven't done any testing yet so far is this part here that does some logic if it's an not an island where it does the wall treatment costs. And so the next obvious unit test that we would write would be for this part of the method. Hopefully, you can see that the refactored version of the code now has a bunch of different seams and different places where we can add different kinds of unit tests. A lot of those unit tests are going to check failure conditions and make sure that things fail in the ways we expect them to with invalid results or occasionally exceptions or forbidden results in case the user is not able to access a certain thing. And then we'll also, of course, want to have a variety of different tests that show that the different happy paths are calculated correctly. Now the trick is that we want to make sure that all these different individual pieces that we can test one by one all work together and that not just our code works, but also the way our code interacts with other dependencies like databases, files, and third-party APIs, that all those things work too. We'll see how that works in the next section.

The Importance of Integration Testing

While anybody who knows me knows that I'm a huge fan of unit tests in part because of the benefits unit testability affords, integration tests are still very important. Just because you have a bunch of parts that do what they're each designed to do, that doesn't mean that they've been assembled properly or that they will fit together with the third-party dependencies that you hope they'll work with. Integration tests ensure your system works with its dependencies and that its internal modules work with one another. There are many different kinds of integration tests, but these are two of the most important ones. The first kind of test ensures that some integration you have with an out-of-process dependency works properly. Can you write to a database or a file? Can you read a value back? Does your integration with a third-party API work properly? The second kind of test is critical if you're following SOLID principles. You're going to have a lot of small modules, and you need to ensure they work together properly. Now a common question that I see from clients is should I unit test or integration test my ASP.NET Core controllers or endpoints? ASP.NET integration tests, which use the `WebApplicationFactory`, have a lot of nice characteristics. They are nearly as fast as unit tests. They're more resilient to refactoring. And by that I mean, if you refactor your endpoints so that things that are currently inside the endpoint are moved into cross-cutting concerns using filters or something like that, your integration tests will continue to work as long as the external behavior of that endpoint doesn't change. Your unit tests will break. When you're using integration tests, you can include testing for routes, middleware, filters, model binding, and model validation. And because of all these reasons, I strongly prefer to use integration tests with `WebApplicationFactory` over unit tests for your ASP.NET Core controllers or endpoints.

Demo: Integration Testing SOLID Code

Let's review a couple of integration tests for our refactored system. We'll look at a couple that talk to external infrastructure concerns and another that deals with integrating everything in the application. I've added a `MegaPricer.IntegrationTests` project to the solution. You'll find the source in module 6. Here, I've added a few different types of integration tests. Let's start with our `PriceReport` strategy integration test. Recall that this particular system under test is responsible for creating a CSV file. All this integration test does is verify that if I use `PriceReportCalculationStrategy`, it will actually create an `Orders.csv` file. And you can see the test passes. Looking at the `PriceReportCalculationStrategy`, the

thing we're testing is the Create method shown here where it is configuring a StreamWriter and then writing a few lines to the file. And then once the thing is disposed, the file will be written. It has another method called AddFeatureLine. We can test this one as well. We're still going to create the file as part of this test, so you'll see that on line 18 here. And then we're going to call AddFeature. And in order to verify that we added the line successfully, we're going to specify that the SKU for the feature is a GUID string. That way, we can just read back all the text from the file and verify that it does, in fact, include the string that we are looking for. And you can see this test passes as well. Another thing that we would like to test is that we can write to our database successfully. And so the `SqliteOrderDataService_CreateNewOrder` class has a method in there that says it should add a new order to the database. We set this up in its constructor to connect to a SQLite database and ensure that it's created. Then the test itself just creates a new order. We're going to use our `SqliteOrderDataService`. Notice that there's no mocking going on here. These are real services talking to real infrastructure. We're going to create the order, and then we're going to assert that it's true that the order exists. And we're going to figure that out by making a fresh query to a `DbContext` to see if that order is in the database. And you can see this test passes. Our last test is one that is going to test the entire system, including the web application front end. So, this test is inside of `HomePage_OnGet` as the class, and the method name is `IncludesExpectedPrices`. So, you've seen throughout this course, from time to time, I've run the application, and you've seen a few prices that are included on that home page. This test is going to verify that in an automated fashion. One challenge with many ASP.NET Core applications is authentication. You'll see that this test also takes that into account. First, we get a client that works with our factory. Notice that we are ensuring that cookies are being handled, but we are not automatically redirecting. Next, we're going to go and fetch the login page. And from there, we're going to pull out the `RequestVerificationToken`. We need this when we submit to the login handler because it's going to look for that token to ensure that this isn't a cross-site forgery attack. We construct our payload as a dictionary, encode it using `FormUrlEncodedContent`, and then post it here inside the breakpoint where you see `client.PostAsync`. Once we get back that response, we can verify that it is successful if we want to or inspect it in the debugger, which is what I was doing prior to now, and that's why there was a breakpoint. But once all of that is working, then we are ready to make our simple request to the home page by calling `GetAsync` on the `/` route. Once again, we'll just take all the content and read it into a string, and then we can assert that that string contains the values that we're

expecting to have from that page. As you can see, the test passes. And of course, if we were going to be doing a lot of these types of tests, we would extract out all the login logic into a helper method of some sort so that we wouldn't have to copy it into each of our individual tests. Now, we can run all the tests for this whole solution. And you'll see that in each test project, the first test always takes a little bit longer because of the project startup. This is especially true for integration tests. But after that, they tend to run fairly quickly. So, you can see the `HomePage_OnGet` took about a second. That includes all the startup for the web host and the test project. And then after that, they were in the double digit milliseconds. The first unit test took 39 ms or so. And then after that, they were very quick with a few of the other pricing service ones taking 58, 54 ms. But again, very, very fast sets of tests that you should be able to run hundreds of these very frequently for the unit tests especially. And even these integration tests are not terribly slow for what we're trying to do here. There's really no excuse for not including these, at least as part of your CI build because they're going to give you a safety net that you need, especially as you're refactoring the code or in the future when you're upgrading it to newer versions of .NET or newer versions of dependencies that you may have.

Next Steps

If you've been watching this course pretty much straight through, hopefully you have a decent understanding of the system we've been working on in mind. Either way, you may want to pull up the code from the latest demos file for this step. I want you to think about the system as it stands now and come up with one or two things you would do to improve its design. If you had the time and if it was deemed important enough, these are the next things that you would do. Pause the video for a moment and think about it. When you're ready, resume the video. Okay, hopefully you have a couple of ideas. It's totally fine if they don't align with mine, but here's what I would be thinking about. First, I would introduce a proper object-relational mapper like EF Core or Dapper. You saw that I already have EF Core there behind the scenes, but it's only being used to facilitate me setting up the sample data and the original schema, not for anything the actual system is using. The current system still has a lot of low-level code for talking to databases and then mapping those results to the model types. Remember, we didn't get rid of all those data tables. We just mapped them inside of helper methods. So, we could make the code faster, leaner, and easier to work with by replacing the current data service implementations with a more advanced tool for querying and mapping data. Next, I'd write

more unit tests. I haven't bothered calculating test coverage for the system at this point because I know it's pretty poor. Also, test coverage isn't really a great metric to focus on, but it can help if you're not sure which parts of your mission-critical code are completely untested. More unit tests will also increase our confidence when making additional refactoring changes, as well as when we add additional features. And that brings us to my third step, which would be to enhance the model types with more behavior. Right now, the types we extracted are still mostly being operated on rather than performing operations on their own data. That's a violation of the Tell, Don't Ask principle. And it's one that I would correct over time by making the state of the objects mostly read-only and by exposing methods for modifying the state of the objects or having them perform operations. This allows for much better encapsulation, which reduces coupling between objects and eliminates duplication. It also helps by making implicit operations explicit. How does that stack up with the things that you would do? If you have a moment, open up that demos file and see if you can modify the system by adding a couple of unit tests or enhancing it in some of the ways I've described here or that you have come up with. You'll find that you'll get much better at refactoring the more you actually practice refactoring as opposed to just watching someone else go through the steps.

Summary

In this module, we evaluated the progress we've made with the legacy software system we've worked on throughout this course. We saw how the code has improved, both from a qualitative standpoint based on our opinions and feelings about the code, as well as from a quantitative standpoint using key code metrics. Next, we looked at the concept of unit testability and the importance of unit tests for the system, and we saw a few examples. And of course, unit tests are great, but they only go so far. It's also important to have some automated integration tests that prove your system works properly with its dependencies and that its internal components work correctly with one another. And finally, we considered next steps for improving the system. No software system that is being used is ever really done. And in addition to more features, there are certainly still things we could do to improve the internal quality of this system. Hopefully, you came up with a few ideas yourself, and I shared a few things I would try to address given the time. And that brings us to next steps for you. I hope this course has been helpful and you've learned a few new techniques for tackling complex legacy code by applying SOLID principles. Please leave some feedback if you have a moment. If you haven't watched

my other SOLID or refactoring courses, I would encourage you to do so. You'll find the complete list on my author page. And in most cases, even the retired courses have more and different content than the newer ones. That's because the more recent courses often needed to be shorter. If you're good with SOLID and refactoring now, then my recommended next step for you is either domain-driven design or to check out some additional design patterns, both of which will give you even more ideas for ways to tackle complex problems and improve your designs. Until next time, keep improving.